Lab 5: Using Azure Key Vault Secrets in a Databricks Notebook

Description

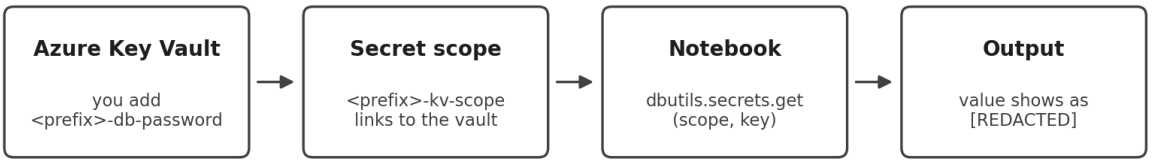
You will store a secret in Azure Key Vault, link the vault to Databricks with a secret scope, and read the secret in a notebook without ever typing the value into code. You will then let your Data Engineers group read the scope and confirm that your Data Analysts cannot.

Learning Objectives

- Add a secret to Azure Key Vault
- Create an Azure Key Vault-backed secret scope
- Read a secret in a notebook and see how redaction works
- Grant READ on a scope to a group and verify it with test users

Step by Step Guide

How it works



No secret is copied into Databricks. The scope's permissions decide who can read it: READ for data-engineers, nothing for analysts.

Your lab values

Item	Value
Prefix	Same as earlier labs (for example bob)
Key Vault	Name, Vault URI and Resource ID from your trainer
Secret name	<prefix>-db-password
Secret scope	<prefix>-kv-scope

Item	Value
Groups and test users	From Lab 1: <prefix>-data-engineers, <prefix>-data-analysts, <prefix>-engineer1, <prefix>-analyst1

Important: Only use made-up values in this lab, never a real password.

Step 1: Add a secret to Key Vault

1. In the Azure portal, open the Key Vault named by your trainer > Objects > Secrets > Generate/Import.
2. Name: <prefix>-db-password. Secret value: a made-up value such as Lab5-Demo-Pass!. Click Create.

> Check: Your secret appears in the list with status Enabled. You will also see other learners' secrets, because everyone shares this vault.

Step 2: Create your secret scope

1. In a new browser tab, open your workspace URL and add #secrets/createScope to the end, for example https://adb-123.azuredatabricks.net/#secrets/createScope. The capital S matters.
2. Scope Name: <prefix>-kv-scope.
3. Manage Principal: Creator. Only you can manage the scope.
4. DNS Name: the Vault URI from your trainer. Resource ID: the vault's Resource ID.
5. Click Create.

> Check: A message confirms the scope was created. No secret has been copied; the scope is only a link to the vault.

Step 3: Read the secret in a notebook

1. In the workspace, click New > Notebook. Keep the language as Python and connect to Serverless compute.
2. Run each block in its own cell:

```
# 1. List the scopes you can see, and the secret names in yours
print(dbutils.secrets.listScopes())
print(dbutils.secrets.list("<prefix>-kv-scope"))
```

```
# 2. Read the secret. Only the scope and key names are in the code.
pw = dbutils.secrets.get(scope="<prefix>-kv-scope", key="<prefix>-db-password")
print(pw)
print(len(pw))
```

```
# 3. Use it the way a real connection string would
conn = f"Server=sql-demo;User=etl;Password={pw};"
print(conn)
```

> Check: The list shows every secret in the shared vault. print(pw) shows [REDACTED], the length is the real length, and the connection string also shows [REDACTED] where the password is.

Note: Redaction only stops accidental display. Anyone who can read the scope could still expose the value on purpose, so READ access must be given carefully.

Step 4: Rotate the secret in one place

1. In Key Vault, open <prefix>-db-password > New Version. Enter a longer made-up value and click Create.
2. Re-run cell 2, the one that reads the secret.

> Check: The length changes. The notebook picked up the new version without any code change.

Step 5: Give your Data Engineers READ on the scope

In a new cell, run:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.workspace import AclPermission

w = WorkspaceClient()
scope = "<prefix>-kv-scope"

w.secrets.put_acl(scope=scope,
                  principal="<prefix>-data-engineers",
                  permission=AclPermission.READ)

for acl in w.secrets.list_acls(scope=scope):
   print(acl.principal, acl.permission)
```

> Check: You are listed with MANAGE and <prefix>-data-engineers with READ. The analysts group is not listed, so it has no access.

Step 6: Verify as the Data Engineer

1. Open a private browser window and sign in to the workspace as <prefix>-engineer1@saurabhuptut.onmicrosoft.com.
2. Create a new Python notebook on Serverless compute and run:

```
pw = dbutils.secrets.get(scope="<prefix>-kv-scope", key="<prefix>-db-password")
print(pw, len(pw))
```

> Check: The engineer sees [REDACTED] and the length. They can use the secret without ever seeing it.

Step 7: Verify as the Data Analyst

Analysts only have SQL access (Lab 1), so they check in SQL Editor.

1. Open a new private window and sign in as <prefix>-analyst1@saurabhuptut.onmicrosoft.com.
2. Open SQL Editor, select the shared SQL warehouse and run:

```
SELECT secret('<prefix>-kv-scope', '<prefix>-db-password');
```

> Check: The query fails with a permission error, because the analysts group has no READ on your scope.

Outcomes

- A secret is stored in Key Vault and linked to Databricks through your own scope
- Your notebook reads the secret by name, and the value shows as [REDACTED]
- Rotating the secret in Key Vault needed no code change
- Data Engineers can read the scope; Data Analysts cannot